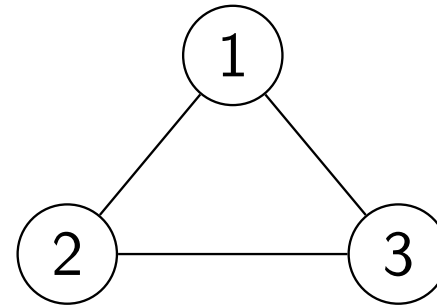
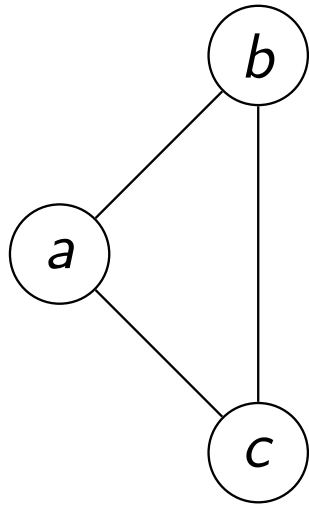


Computing the Graph Edit Distance

Jonas Handwerker

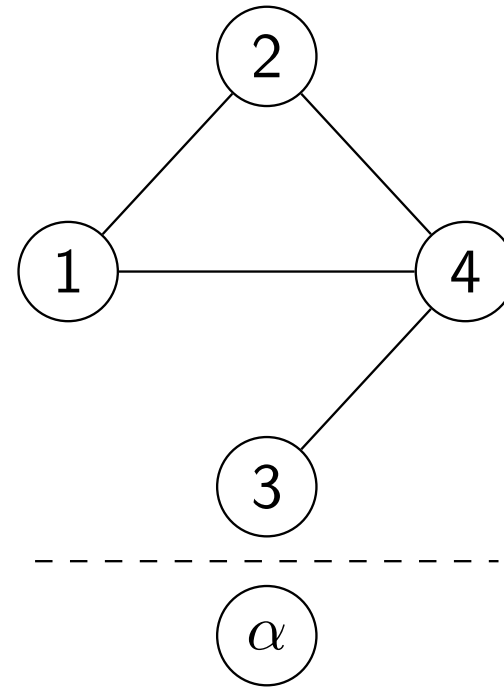
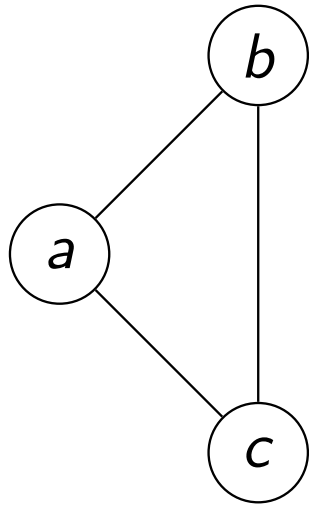
16th September 2026

The Graph Edit Distance



Graph Isomorphism: Are the graphs equivalent?

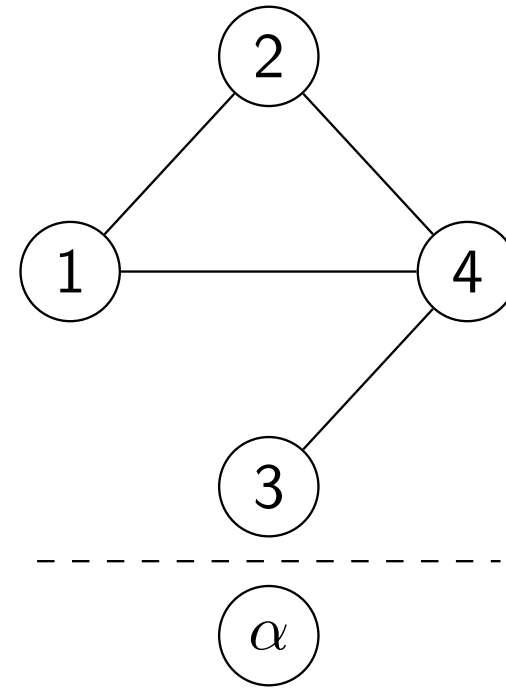
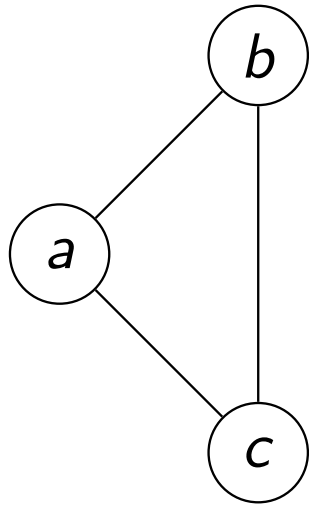
The Graph Edit Distance



Graph Isomorphism: Are the graphs equivalent?

What if graphs vary slightly?

The Graph Edit Distance



Graph Isomorphism: Are the graphs equivalent?

What if graphs vary slightly?

- Quantify difference with the **Graph Edit Distance**

Edit Paths

Definition (Edit Path, [Das+25])

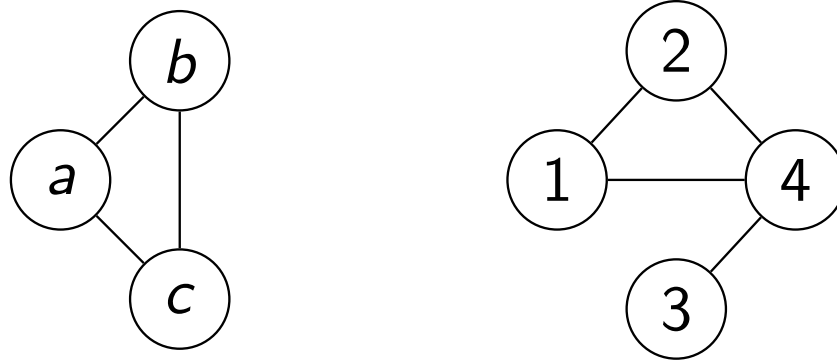
Let G and H be two undirected graphs where each node and edge has a label. An **edit path** from graph G to graph H is a sequence

$$(G, o_1, G^1, o_2, \dots, o_k, G^k),$$

where G^k is isomorphic to H and G^k is derived from G^{k-1} by applying the **edit operation** o_k . An edit operation consists of

- ▶ insertion or deletion of an isolated node
- ▶ insertion or deletion of an edge
- ▶ relabeling of a node or edge

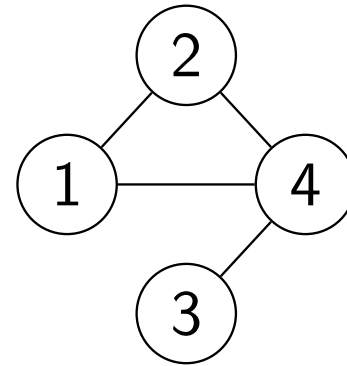
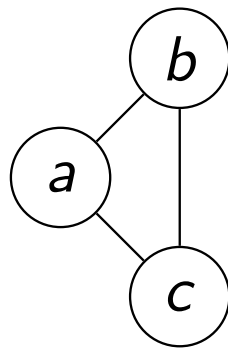
Edit Paths



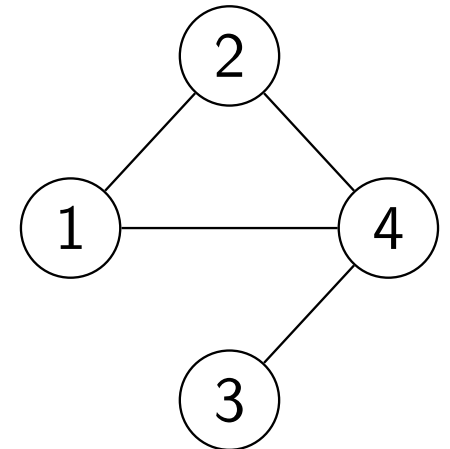
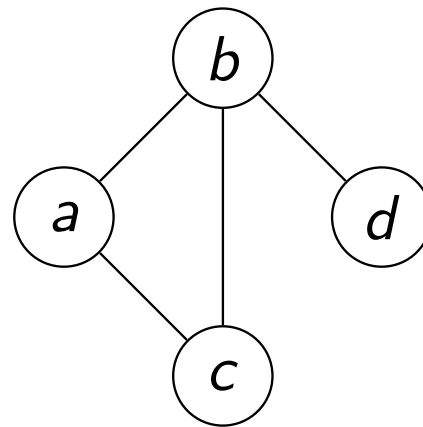
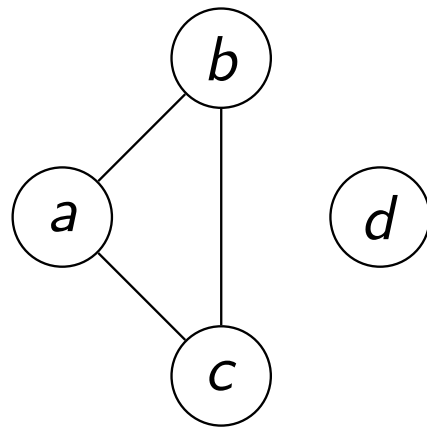
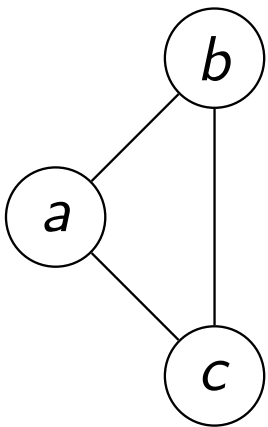
Graphs G and H

What is an edit path between G and H ?

Edit Paths

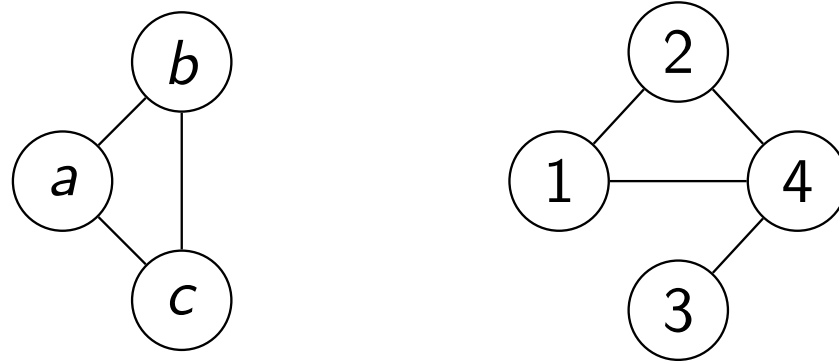


Graphs G and H

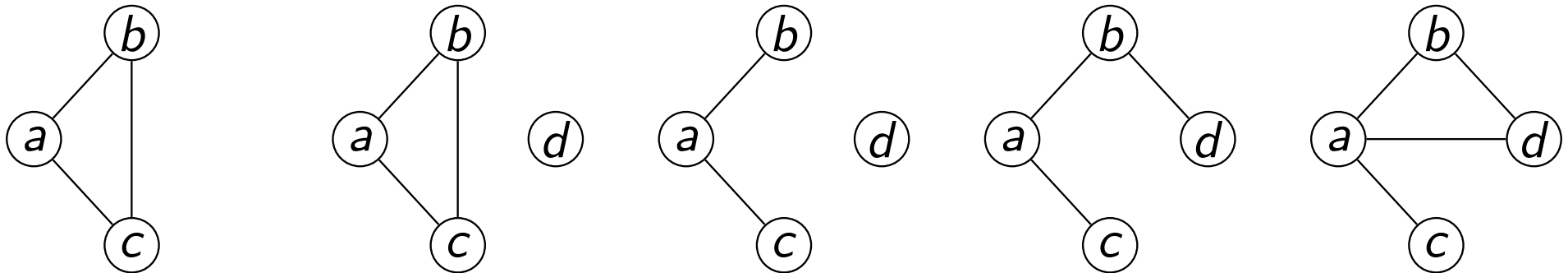


An edit path between G and H

Edit Paths



Graphs G and H



Another edit path between G and H

Graph Edit Distance

Definition (Graph Edit Distance, [Das+25])

Given nonnegative costs c_V, c_E for the edit operations, find an edit path

$$P = (G, o_1, G^1, o_2, \dots, o_k, G^k)$$

from G to H minimizing $\sum_{i=1}^k c(o_i)$. We call the minimum the **graph edit distance** $\text{GED}(G, H)$.

Node Maps

Definition (Node Map, [Das+25])

Let $V_{G+\epsilon}$ be the vertex set of G augmented by the dummy node ϵ_G .
Similarly $V_{H+\epsilon}$.

A node map $\pi \subset V_{G+\epsilon} \times V_{H+\epsilon}$ is a relation in which

- ▶ each node $v \in V_G$ is mapped to exactly one element of $V_{H+\epsilon}$
- ▶ each node $w \in V_H$ has exactly one preimage in $V_{G+\epsilon}$

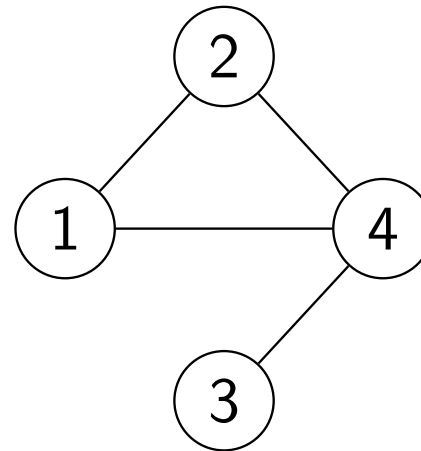
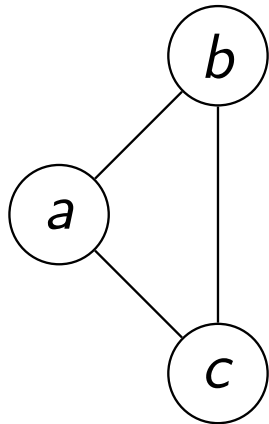
Node Maps

Definition (Node Map, [Das+25])

Let $V_{G+\epsilon}$ be the vertex set of G augmented by the dummy node ϵ_G . Similarly $V_{H+\epsilon}$.

A node map $\pi \subset V_{G+\epsilon} \times V_{H+\epsilon}$ is a relation in which

- ▶ each node $v \in V_G$ is mapped to exactly one element of $V_{H+\epsilon}$
- ▶ each node $w \in V_H$ has exactly one preimage in $V_{G+\epsilon}$



Graphs G and H

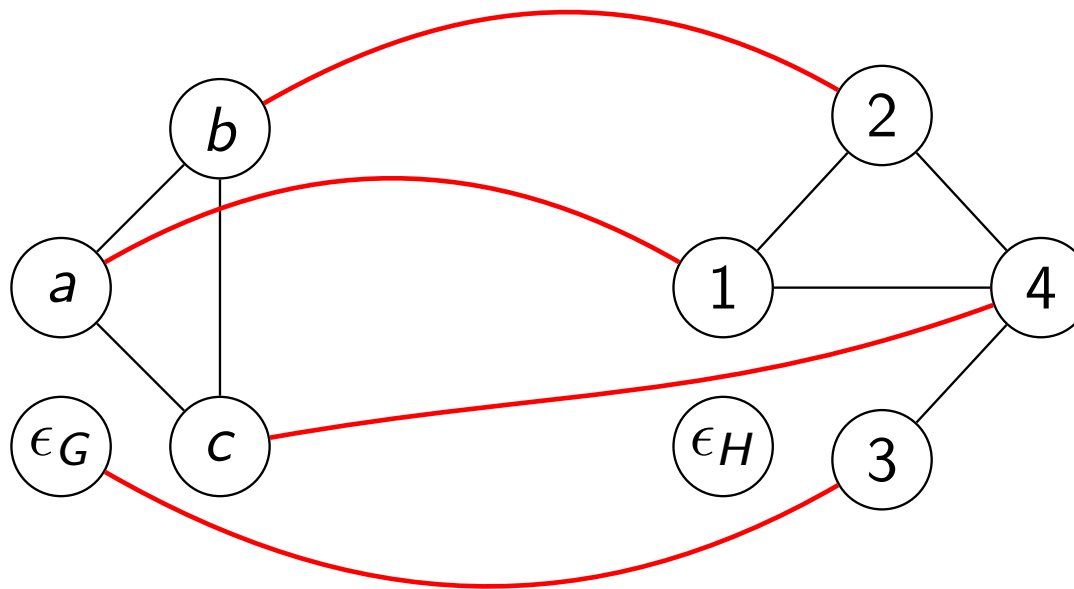
Node Maps

Definition (Node Map, [Das+25])

Let $V_{G+\epsilon}$ be the vertex set of G augmented by the dummy node ϵ_G . Similarly $V_{H+\epsilon}$.

A node map $\pi \subset V_{G+\epsilon} \times V_{H+\epsilon}$ is a relation in which

- ▶ each node $v \in V_G$ is mapped to exactly one element of $V_{H+\epsilon}$
- ▶ each node $w \in V_H$ has exactly one preimage in $V_{G+\epsilon}$



Graphs G and H

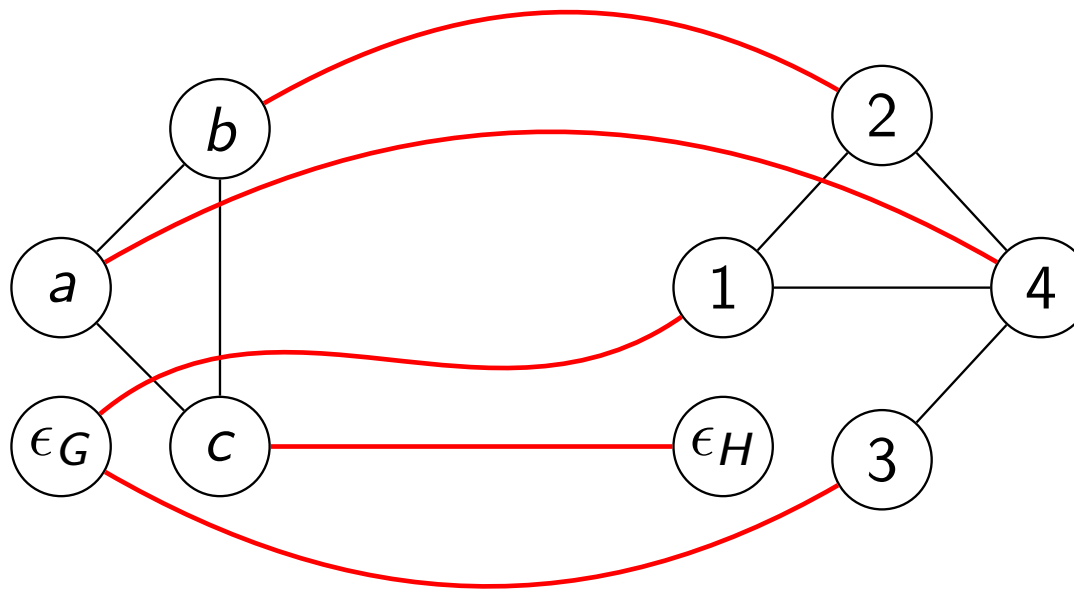
Node Maps

Definition (Node Map, [Das+25])

Let $V_{G+\epsilon}$ be the vertex set of G augmented by the dummy node ϵ_G . Similarly $V_{H+\epsilon}$.

A node map $\pi \subset V_{G+\epsilon} \times V_{H+\epsilon}$ is a relation in which

- ▶ each node $v \in V_G$ is mapped to exactly one element of $V_{H+\epsilon}$
- ▶ each node $w \in V_H$ has exactly one preimage in $V_{G+\epsilon}$



Graphs G and H

Reformulating GED with Node Maps

Theorem ([HJ06])

Given metric edit costs c , finding the minimum GED is equivalent to finding a node map π minimizing induced edit costs.

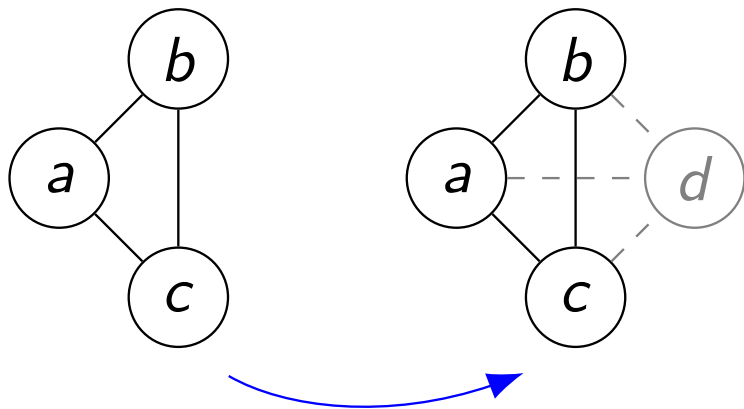
Reformulating GED with Node Maps

Theorem ([HJ06])

Given metric edit costs c , finding the minimum GED is equivalent to finding a node map π minimizing induced edit costs.

Proof:

- project G into a larger, complete graph G_Ω



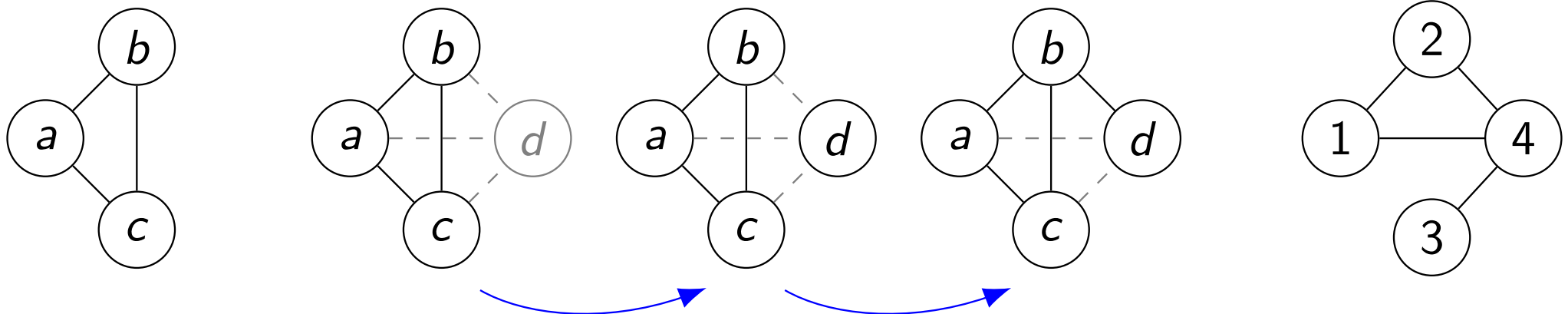
Reformulating GED with Node Maps

Theorem ([HJ06])

Given metric edit costs c , finding the minimum GED is equivalent to finding a node map π minimizing induced edit costs.

Proof:

- ▶ project G into a larger, complete graph G_Ω
- ▶ edit paths correspond to sequences of relabeling operation on G_Ω



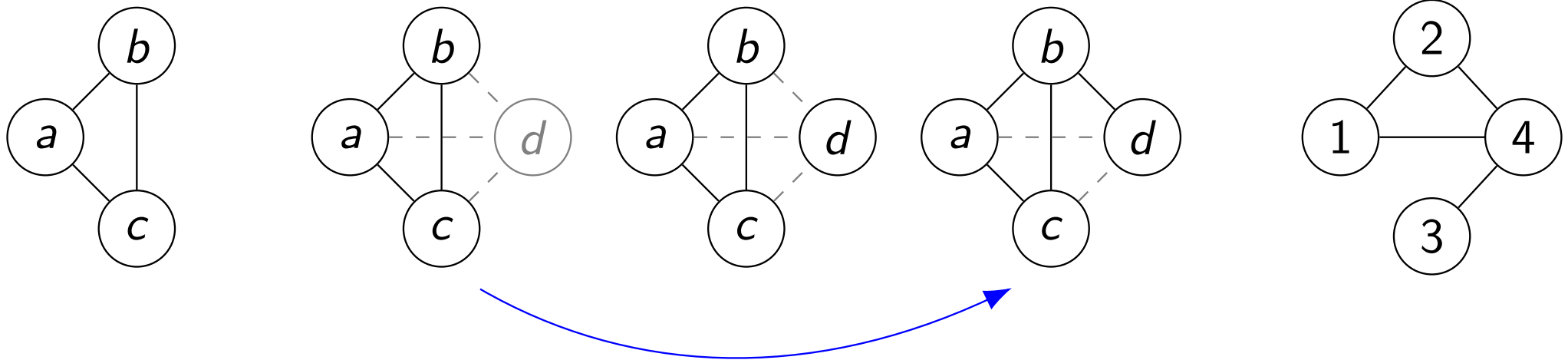
Reformulating GED with Node Maps

Theorem ([HJ06])

Given metric edit costs c , finding the minimum GED is equivalent to finding a node map π minimizing induced edit costs.

Proof:

- ▶ project G into a larger, complete graph G_Ω
- ▶ edit paths correspond to sequences of relabeling operation on G_Ω
- ▶ Use the triangle inequality to aggregate many relabelings into a single step with lower cost



Reformulating GED with Node Maps

Theorem ([HJ06])

Given metric edit costs c , finding the minimum GED is equivalent to finding a node map π minimizing induced edit costs.

Proof:

- ▶ project G into a larger, complete graph G_Ω
- ▶ edit paths correspond to sequences of relabeling operation on G_Ω
- ▶ Use the triangle inequality to aggregate many relabelings into a single step with lower cost

Theorem ([BG20])

A general edit cost function c can be transformed into an equivalent metric cost function.

Formulation F1

Introduce binary variable $x_{i,j}$ for $i \in V_{G+\epsilon}, j \in V_{H+\epsilon}$ to indicate mapping between nodes. Variables $y_{ij,kl}$ indicate edge mappings.

Formulation F1

Introduce binary variable $x_{i,j}$ for $i \in V_{G+\epsilon}, j \in V_{H+\epsilon}$ to indicate mapping between nodes. Variables $y_{ij,kl}$ indicate edge mappings.

$$\min \quad c_1^t x + c_2^t y$$

s.t.

$$x \in \{0, 1\}^{xcard_{F1}}, y \in \{0, 1\}^{ycard_{F1}}$$

Formulation F1

Introduce binary variable $x_{i,j}$ for $i \in V_{G+\epsilon}, j \in V_{H+\epsilon}$ to indicate mapping between nodes. Variables $y_{ij,kl}$ indicate edge mappings.

$$\begin{aligned} \min \quad & c_1^t x + c_2^t y \\ \text{s.t.} \quad & x_{i,\epsilon} + \sum_{k \in V_H} x_{i,k} = 1 & \forall i \in V_G \\ & x_{\epsilon,k} + \sum_{i \in V_G} x_{i,k} = 1 & \forall k \in V_H \\ & y_{ij,\epsilon} + \sum_{kl \in E_H} y_{ij,kl} = 1 & \forall ij \in E_G \\ & y_{\epsilon,kl} + \sum_{ij \in E_G} y_{ij,kl} = 1 & \forall kl \in E_H \end{aligned} \quad \left. \vphantom{\begin{aligned} \text{s.t.} \quad} \right\} \text{Node and edge mappings are valid}$$

$$x \in \{0, 1\}^{xcard_{F1}}, y \in \{0, 1\}^{ycard_{F1}}$$

Formulation F1

Introduce binary variable $x_{i,j}$ for $i \in V_{G+\epsilon}, j \in V_{H+\epsilon}$ to indicate mapping between nodes. Variables $y_{ij,kl}$ indicate edge mappings.

$$\min \quad c_1^t x + c_2^t y$$

$$\text{s.t.} \quad x_{i,\epsilon} + \sum_{k \in V_H} x_{i,k} = 1 \quad \forall i \in V_G$$

$$x_{\epsilon,k} + \sum_{i \in V_G} x_{i,k} = 1 \quad \forall k \in V_H$$

$$y_{ij,\epsilon} + \sum_{kl \in E_H} y_{ij,kl} = 1 \quad \forall ij \in E_G$$

$$y_{\epsilon,kl} + \sum_{ij \in E_G} y_{ij,kl} = 1 \quad \forall kl \in E_H$$

Node and edge mappings are valid

$$y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall ij \in E_G, kl \in E_H$$

$$y_{ij,kl} \leq x_{i,l} + x_{j,l} \quad \forall ij \in E_G, kl \in E_H$$

topological constraints

$$x \in \{0, 1\}^{x_{\text{card}}_{F1}}, y \in \{0, 1\}^{y_{\text{card}}_{F1}}$$

Formulation F1

Introduce binary variable $x_{i,j}$ for $i \in V_{G+\epsilon}, j \in V_{H+\epsilon}$ to indicate mapping between nodes. Variables $y_{ij,kl}$ indicate edge mappings.

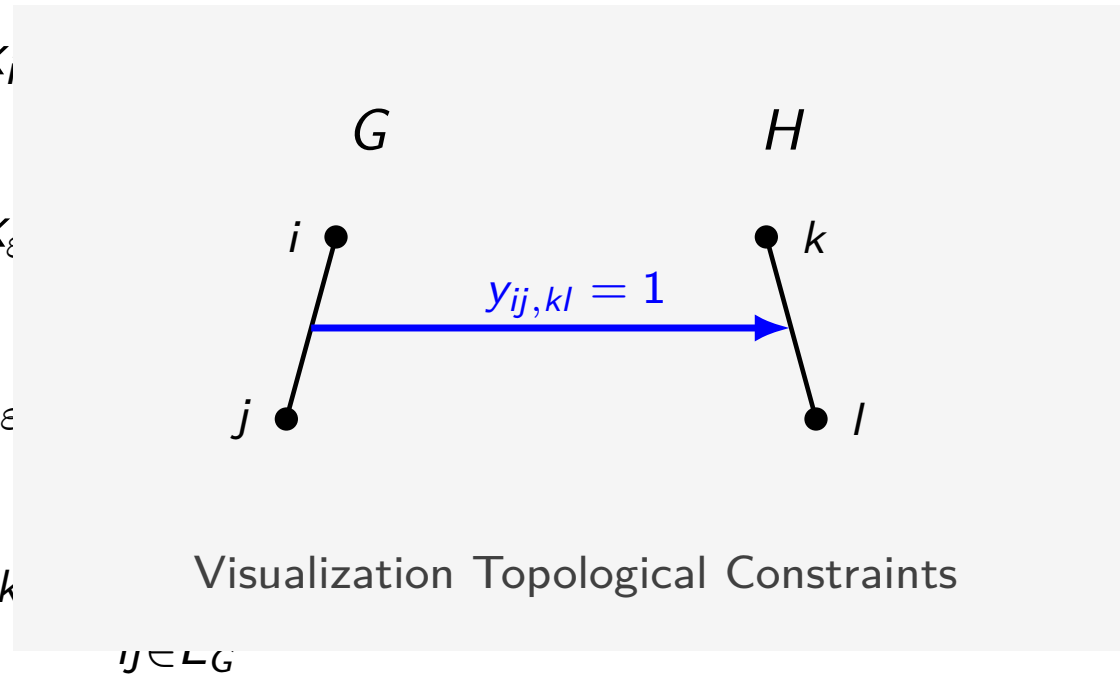
$$\min \quad c_1^t x + c_2^t y$$

s.t. $x_{i,j}$

x_{ϵ}

$y_{ij,\epsilon}$

$y_{\epsilon,k}$



$$\begin{aligned} y_{ij,kl} &\leq x_{i,k} + x_{j,k} & \forall ij \in E_G, kl \in E_H \\ y_{ij,kl} &\leq x_{i,l} + x_{j,l} & \forall ij \in E_G, kl \in E_H \end{aligned} \quad \left. \vphantom{\begin{aligned} y_{ij,kl} &\leq x_{i,k} + x_{j,k} \\ y_{ij,kl} &\leq x_{i,l} + x_{j,l} \end{aligned}} \right\} \begin{array}{l} \text{topological} \\ \text{constraints} \end{array}$$

$$x \in \{0, 1\}^{x_{card_{F1}}}, y \in \{0, 1\}^{y_{card_{F1}}}$$

Formulation F1

Introduce binary variable $x_{i,j}$ for $i \in V_{G+\epsilon}, j \in V_{H+\epsilon}$ to indicate mapping between nodes. Variables $y_{ij,kl}$ indicate edge mappings.

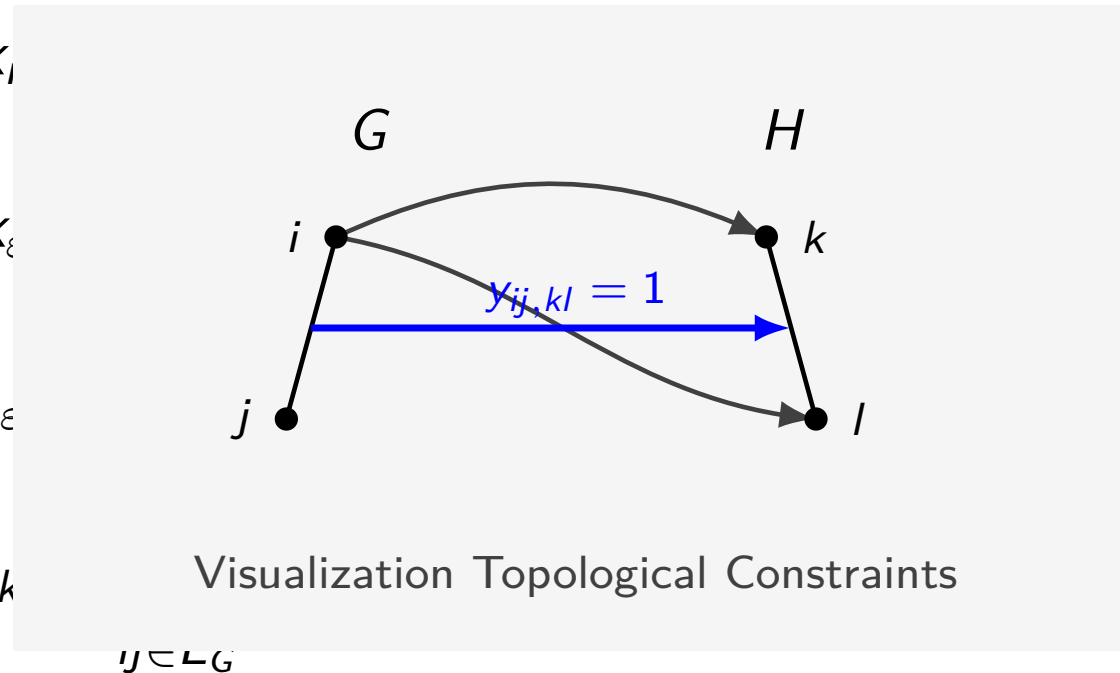
$$\min \quad c_1^t x + c_2^t y$$

s.t. $x_{i,j}$

$x_{i,j}$

$y_{ij,kl}$

$y_{ij,kl}$



$$\left. \begin{aligned} y_{ij,kl} &\leq x_{i,k} + x_{j,k} & \forall ij \in E_G, kl \in E_H \\ y_{ij,kl} &\leq x_{i,l} + x_{j,l} & \forall ij \in E_G, kl \in E_H \end{aligned} \right\} \begin{array}{l} \text{topological} \\ \text{constraints} \end{array}$$

$$x \in \{0, 1\}^{x_{card_{F1}}}, y \in \{0, 1\}^{y_{card_{F1}}}$$

Formulation F1

Introduce binary variable $x_{i,j}$ for $i \in V_{G+\epsilon}, j \in V_{H+\epsilon}$ to indicate mapping between nodes. Variables $y_{ij,kl}$ indicate edge mappings.

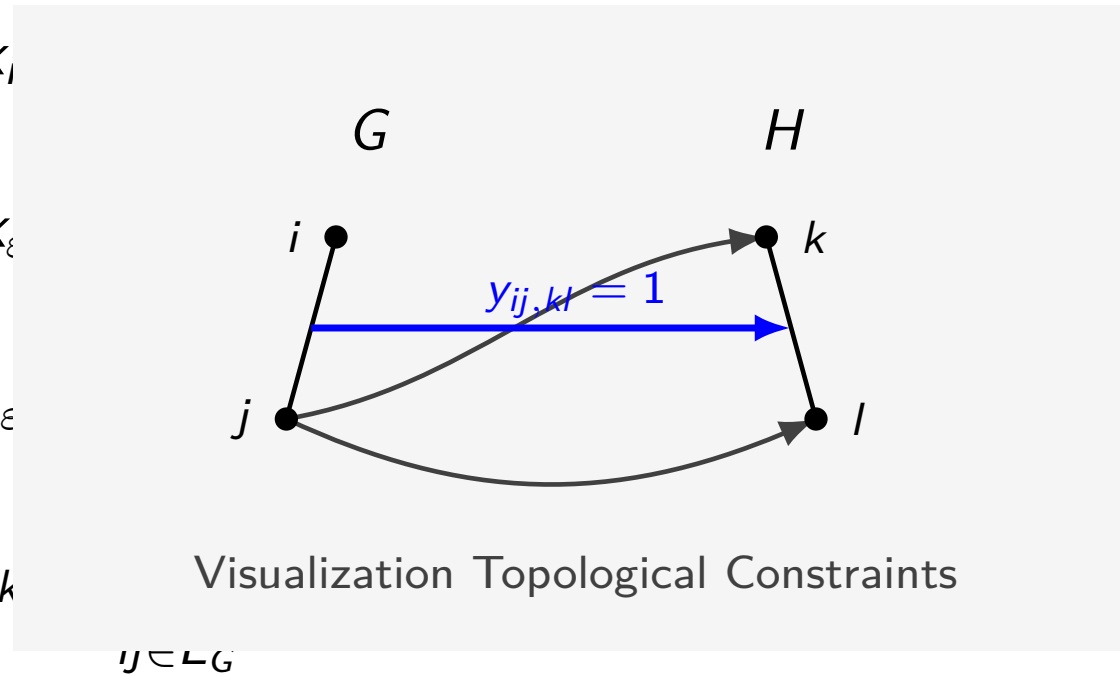
$$\min \quad c_1^t x + c_2^t y$$

s.t. $x_{i,j}$

x_{ϵ}

$y_{ij,\epsilon}$

$y_{\epsilon,k}$



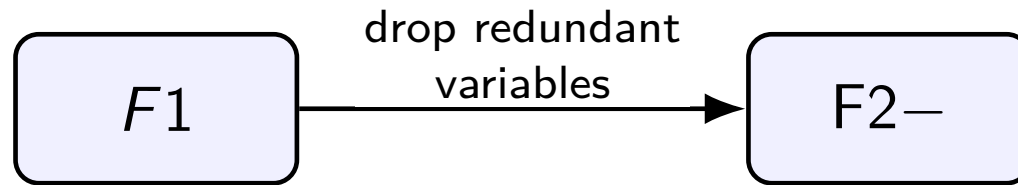
$$\begin{aligned} y_{ij,kl} &\leq x_{i,k} + x_{j,k} & \forall ij \in E_G, kl \in E_H \\ y_{ij,kl} &\leq x_{i,l} + x_{j,l} & \forall ij \in E_G, kl \in E_H \end{aligned} \quad \left. \vphantom{\begin{aligned} y_{ij,kl} &\leq x_{i,k} + x_{j,k} \\ y_{ij,kl} &\leq x_{i,l} + x_{j,l} \end{aligned}} \right\} \begin{array}{l} \text{topological} \\ \text{constraints} \end{array}$$

$$x \in \{0, 1\}^{x_{card_{F1}}}, y \in \{0, 1\}^{y_{card_{F1}}}$$

ILP formulations [Das+25; Ler+17]

$F1$

ILP formulations [Das+25; Ler+17]



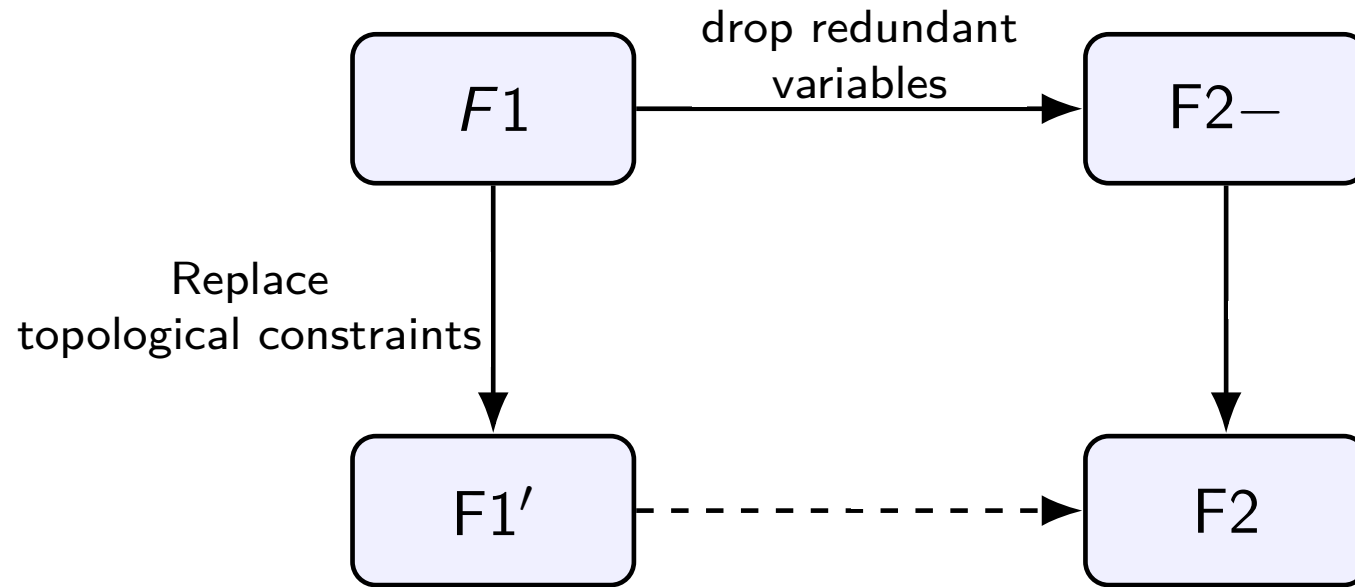
Formulation F2-: removing redundant variables

$$\begin{aligned} \min \quad & c_1^t x + c_2^t y \\ \text{s.t.} \quad & \textcolor{red}{x}_{i,\varepsilon} + \sum_{k \in V_H} x_{i,k} = 1 & \forall i \in V_G \\ & \textcolor{red}{x}_{\varepsilon,k} + \sum_{i \in V_G} x_{i,k} = 1 & \forall k \in V_H \\ & \textcolor{red}{y}_{ij,\varepsilon} + \sum_{kl \in E_H} y_{ij,kl} = 1 & \forall ij \in E_G \\ & \textcolor{red}{y}_{\varepsilon,kl} + \sum_{ij \in E_G} y_{ij,kl} = 1 & \forall kl \in E_H \\ & y_{ij,kl} \leq x_{i,k} + x_{j,k} & \forall ij \in E_G, kl \in E_H \\ & y_{ij,kl} \leq x_{i,l} + x_{j,l} & \forall ij \in E_G, kl \in E_H \\ & x \in \{0,1\}^{x_{\text{card}_{F1}}}, y \in \{0,1\}^{y_{\text{card}_{F1}}} \end{aligned}$$

Formulation F2-: removing redundant variables

$$\begin{aligned}
 & \min \quad \bar{c}_1^t x + \bar{c}_2^t y \\
 & \text{s.t.} \quad \sum_{k \in V_H} x_{i,k} \leq 1 && \forall i \in V_G \\
 & \quad \sum_{i \in V_G} x_{i,k} \leq 1 && \forall k \in V_H \\
 & \quad \sum_{kl \in E_H} y_{ij,kl} \leq 1 && \forall ij \in E_G \\
 & \quad \sum_{ij \in E_G} y_{ij,kl} \leq 1 && \forall kl \in E_H \\
 & \quad y_{ij,kl} \leq x_{i,k} + x_{j,k} && \forall ij \in E_G, kl \in E_H \\
 & \quad y_{ij,kl} \leq x_{i,l} + x_{j,l} && \forall ij \in E_G, kl \in E_H \\
 & \quad x \in \{0, 1\}^{|V_G| \cdot |V_H|}, y \in \{0, 1\}^{|E_G| \cdot |E_H|}
 \end{aligned}$$

ILP formulations [Das+25; Ler+17]



Formulation F2: Improving the topological constraints

Recall

$$y_{ij,kl} \leq x_{i,k} + x_{j,k}$$

$$\forall ij \in E_G, kl \in E_H$$

$$y_{ij,kl} \leq x_{i,l} + x_{j,l}$$

$$\forall ij \in E_G, kl \in E_H$$

Formulation F2: Improving the topological constraints

Recall

$$y_{ij,kl} \leq x_{i,k} + x_{j,k} \qquad \forall ij \in E_G, kl \in E_H$$

$$y_{ij,kl} \leq x_{i,l} + x_{j,l} \qquad \forall ij \in E_G, kl \in E_H$$

We replace these constraints with the implication

Edge ij is mapped to any edge in $\delta(k) \implies \pi(i) = k$ or $\pi(j) = k$.

Formulation F2: Improving the topological constraints

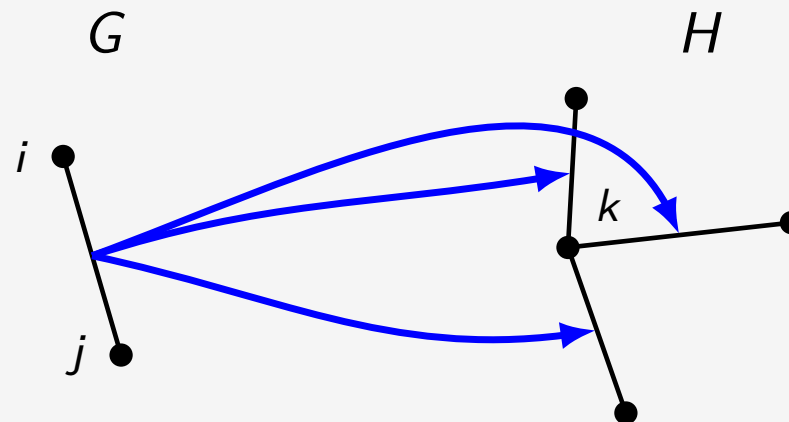
Recall

$$y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall ij \in E_G, kl \in E_H$$

$$y_{ij,kl} \leq x_{i,l} + x_{j,l} \quad \forall ij \in E_G, kl \in E_H$$

We replace these constraints with the implication

Edge ij is mapped to any edge in $\delta(k) \implies \pi(i) = k$ or $\pi(j) = k$.



Visualization Topological Constraints

Formulation F2: Improving the topological constraints

Recall

$$y_{ij,kl} \leq x_{i,k} + x_{j,k}$$

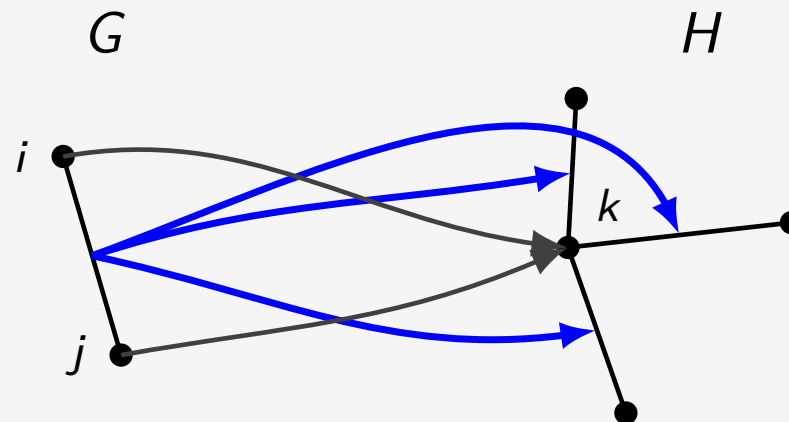
$$\forall ij \in E_G, kl \in E_H$$

$$y_{ij,kl} \leq x_{i,l} + x_{j,l}$$

$$\forall ij \in E_G, kl \in E_H$$

We replace these constraints with the implication

Edge ij is mapped to any edge in $\delta(k) \implies \pi(i) = k$ or $\pi(j) = k$.



Visualization Topological Constraints

Formulation F2: Improving the topological constraints

Recall

$$y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall ij \in E_G, kl \in E_H$$

$$y_{ij,kl} \leq x_{i,l} + x_{j,l} \quad \forall ij \in E_G, kl \in E_H$$

We replace these constraints with the implication

Edge ij is mapped to any edge in $\delta(k) \implies \pi(i) = k$ or $\pi(j) = k$.

As a linear equation

$$\sum_{l \in \Gamma_H(k)} y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall k \in V_H, ij \in E_G.$$

Formulation F2: Improving the topological constraints

Recall

$$y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall ij \in E_G, kl \in E_H$$

$$y_{ij,kl} \leq x_{i,l} + x_{j,l} \quad \forall ij \in E_G, kl \in E_H$$

We replace these constraints with the implication

Edge ij is mapped to any edge in $\delta(k) \implies \pi(i) = k$ or $\pi(j) = k$.

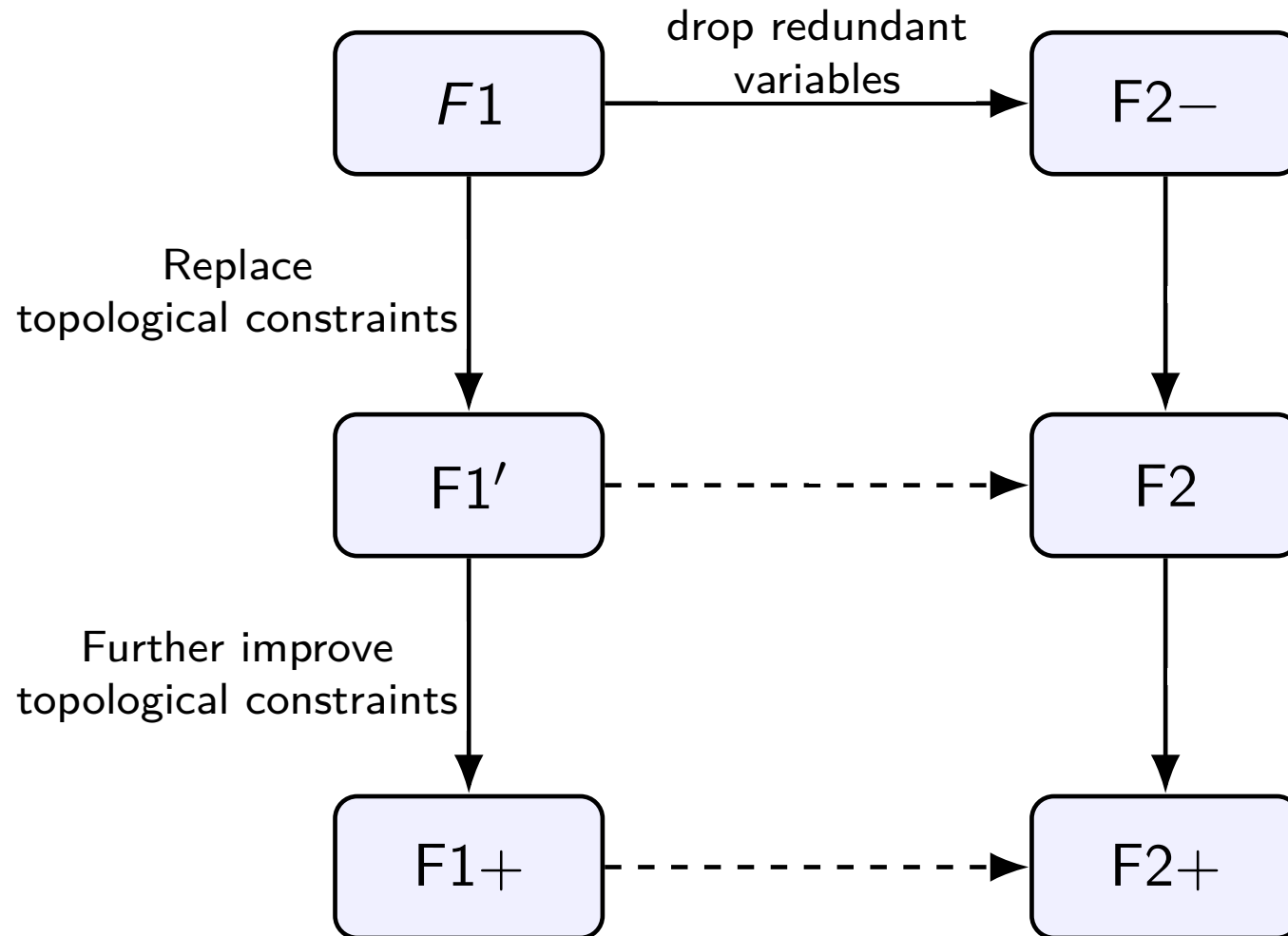
As a linear equation

$$\sum_{l \in \Gamma_H(k)} y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall k \in V_H, ij \in E_G.$$

Side effect: Constraints on edge map variables $y_{ij,kl}$ are now implied.

► reduces number of constraints significantly

ILP formulations [Das+25; Ler+17]



Formulation F2+: Expanding topology constraints

We saw

Edge ij is mapped to any edge in $\delta(k) \implies \pi(i) = k$ or $\pi(j) = k$.

Formulation F2+: Expanding topology constraints

We saw

Edge ij is mapped to any edge in $\delta(k) \implies \pi(i) = k$ or $\pi(j) = k$.

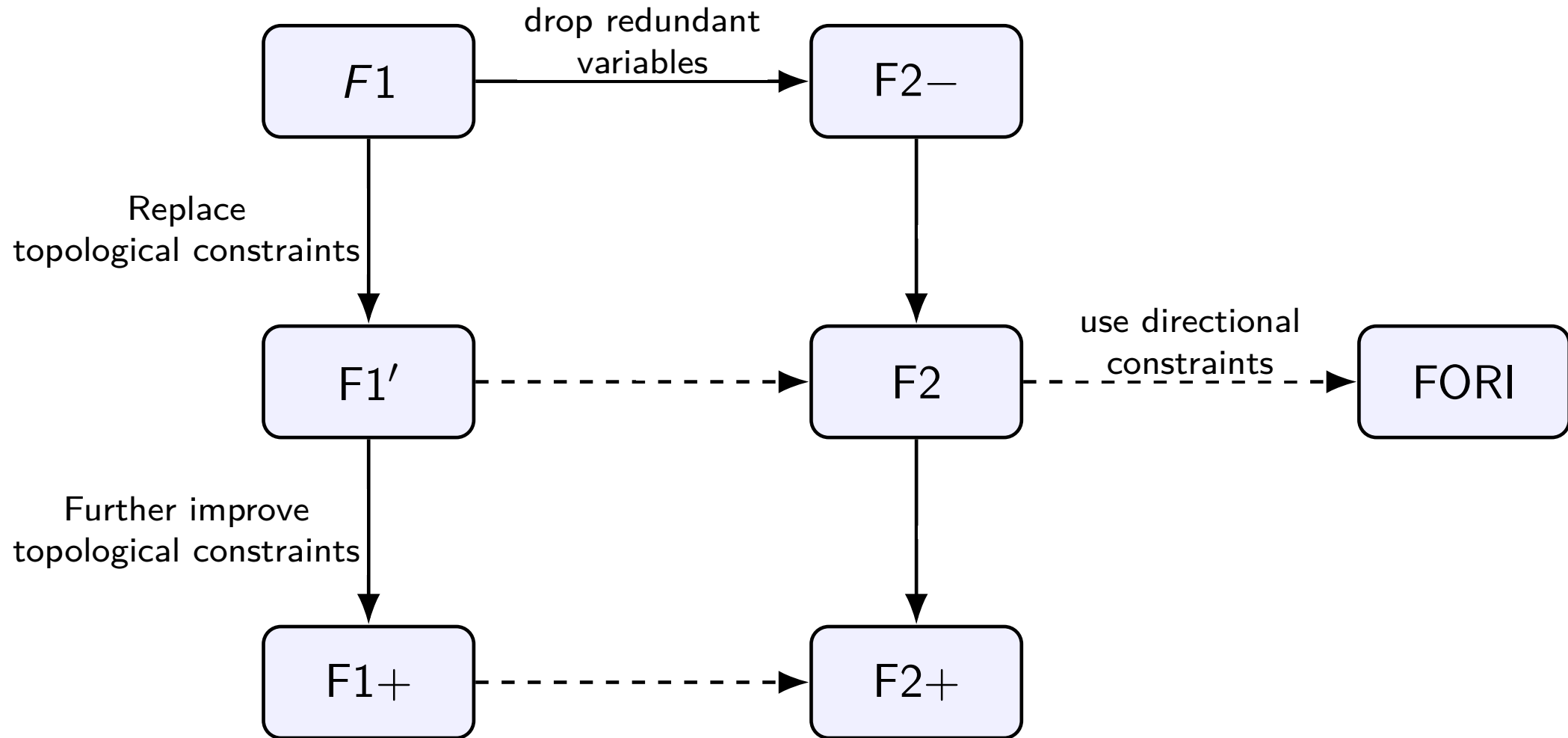
Reverse the roles of G and H to obtain

The pre-image of edge kl is in $\delta(i) \implies \pi(i) = k$ or $\pi(i) = l$.

In a linear equation,

$$\sum_{j \in \Gamma_G(i)} y_{ij,kl} \leq x_{i,k} + x_{i,l} \quad \forall i \in V_G, kl \in E_H.$$

ILP formulations [Das+25; Ler+17]



Formulation FORI: Introducing directionality

Idea:

- ▶ Orient edges i, j in G such that $i < j$ to obtain $\vec{G}$
- ▶ Orient edges in H both ways to obtain $\overleftrightarrow{H}$

Formulation FORI: Introducing directionality

Idea:

- ▶ Orient edges i, j in G such that $i < j$ to obtain $\vec{G}$
- ▶ Orient edges in H both ways to obtain $\overleftrightarrow{H}$

We can now be more precise in topology constraints:

$$\sum_{l \in \Gamma_H(k)} y_{ij,kl} \leq x_{i,k} + x_{j,k} \quad \forall k \in V_H, ij \in E_G.$$

Becomes

$$\sum_{(k,l) \in \delta_{\overleftrightarrow{H}}^+(k)} z_{ij,kl} \leq x_{i,k} \quad \forall k \in V_H, (i,j) \in E_{\vec{G}}$$

$$\sum_{(l,k) \in \delta_{\overleftrightarrow{H}}^-(k)} z_{ij,lk} \leq x_{j,k} \quad \forall k \in V_H, (i,j) \in E_{\vec{G}}$$

Formulation FORI: Introducing directionality

Idea:

- ▶ Orient edges i, j in G such that $i < j$ to obtain $\vec{G}$
- ▶ Orient edges in H both ways to obtain $\overleftrightarrow{H}$

We can now be more precise in topology constraints:

$$\sum_{j \in \Gamma_G(i)} y_{ij,kl} \leq x_{i,k} + x_{i,l} \quad \forall i \in V_G, kl \in E_H$$

Becomes

$$\sum_{(ij) \in \delta_{\vec{G}}^+(i)} z_{ij,kl} + \sum_{(ji) \in \delta_{\vec{G}}^-(i)} z_{ij,lk} \leq x_{i,k} \quad \forall i \in V_G, (kl) \in E_{\overleftrightarrow{H}}$$

Experimental Setup

- ▶ Hardware: AMD Ryzen 7 5800X 4.7GHz 8-core Processor, 32GB 3200MT/s DDR4 RAM
- ▶ Software: Gurobi v13.0.3, 8 threads

Experimental Setup

- ▶ Hardware: AMD Ryzen 7 5800X 4.7GHz 8-core Processor, 32GB 3200MT/s DDR4 RAM
- ▶ Software: Gurobi v13.0.3, 8 threads
- ▶ Solver timeout of 600s
- ▶ For each instance, runtimes are averaged over five fixed seeds
- ▶ An instance counts as solved if at least one run solved the instance

Dataset statistics [Das+25]

AIDS: molecular compounds as labeled graphs

Dataset	min $ V_G $	max $ V_G $	min $ E_G $	max $ E_G $
AIDS				
aids-21-30	22	30	23	30
aids-31-40	32	39	33	42
aids-41-50	43	50	47	55
aids-51-60	52	59	55	65
aids-61-70	62	69	68	75
aids-71-80	73	80	79	96

IMDB: querying on unlabeled graphs

Dataset	$ V_G $	$ E_G $	avg. / max. $ V_Q $	avg. / max. $ E_Q $
imdb-10	89	1467	10 / 10	33 / 45
imdb-20	89	1467	20 / 20	113 / 190
imdb-30	89	1467	30 / 30	279 / 435
imdb-43	89	1467	43 / 43	435 / 719

Runtime Statistics

Runtime results on AIDS. In gray original values

Model	21–30	31–40	41–50	51–60	61–70	71–80
avg. runtime in sec.						
FORI	0.111	0.231	2.975	8.122	4.198	86.202
FORI (paper)	0.20	0.42	5.42	14.73	7.72	99.28
timeouts						
FORI	0	0	0	0	0	3 (6.6%)
FORI (paper)	0	0	0	0	0	3 (6.6%)

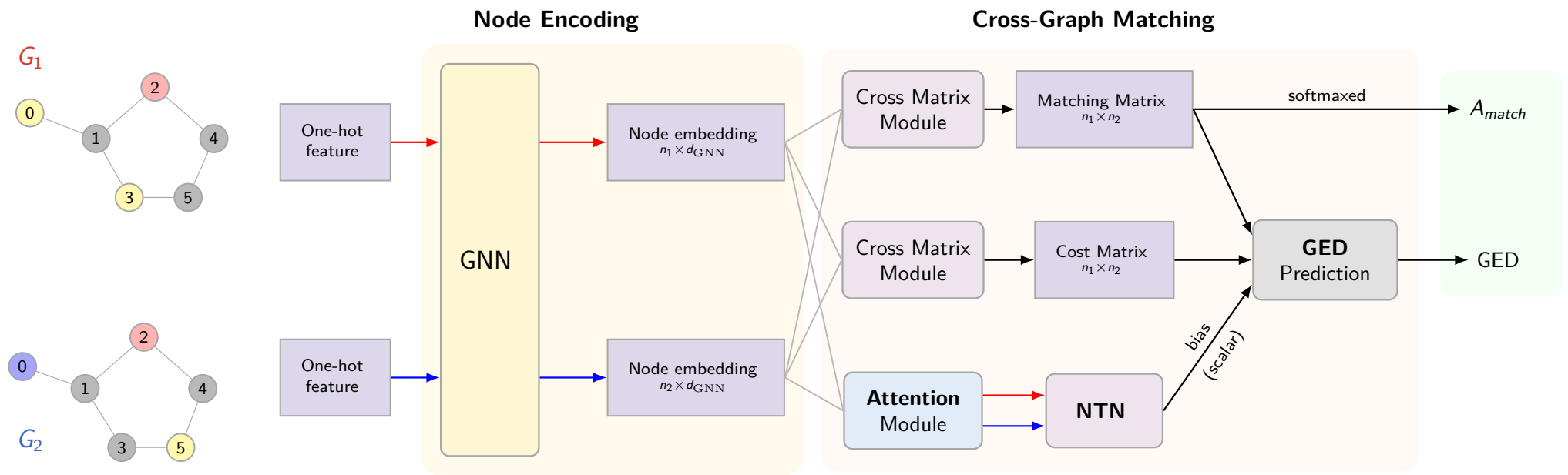
Runtime Statistics

Runtime results on IMDB. In gray original values

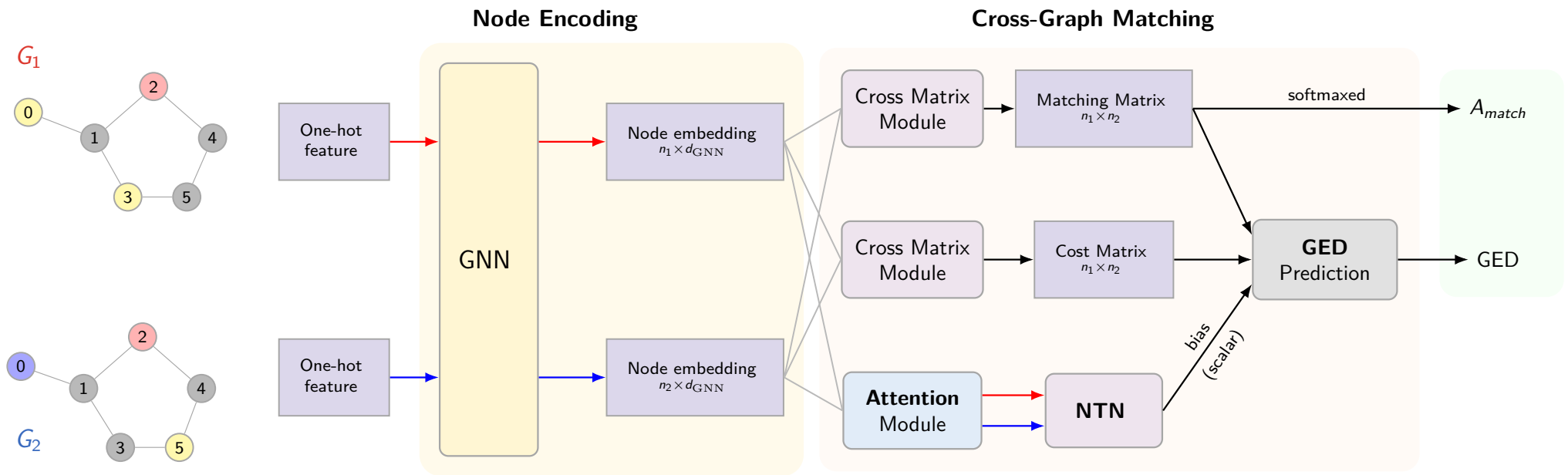
Model	imdb-10	imdb-20	imdb-30	imdb-43
avg. runtime in sec.				
Ours	0.636	2.448	8.225	604.94
FORI (paper)	1.77	8.45	22.95	604.12
timeouts				
Ours	0	0	0	6 (100%) ¹
FORI (paper)	0	0	0	6 (100%)

¹In 2/6 cases, a better solution was found

Structure of GEDGNN [Pia+23]



Structure of GEDGNN [Pia+23]



$$GED = \sigma(\text{Softmax}(A_{match}) \cdot A_{cost} + \text{bias})$$

Generating a Graph Edit Path

Problem: We have no way to know how off the prediction of the model is.

Generating a Graph Edit Path

Problem: We have no way to know how off the prediction of the model is.

Observation

The A_{match} matrix is trained to predict which node pairs are part of some optimal node matching.

Idea: Use **weighted bipartite perfect matching** to compute likely good node matchings.

Generating a Graph Edit Path

Problem: We have no way to know how off the prediction of the model is.

Observation

The A_{match} matrix is trained to predict which node pairs are part of some optimal node matching.

Idea: Use **weighted bipartite perfect matching** to compute likely good node matchings.

⇒ gives a tangible upper bound on GED*

Generating a Graph Edit Path

Let G be the complete bipartite graph with

$$V(G) = V_1 \dot{\cup} V_2, \quad c(\{v_i, u_j\}) = A_{match}^{ij}$$

We can find the maximum weight (perfect) matching in G in time $O(n^3)$ ².

²Hungarian Method [Kuh55]

Generating a Graph Edit Path

Let G be the complete bipartite graph with

$$V(G) = V_1 \dot{\cup} V_2, \quad c(\{v_i, u_j\}) = A_{match}^{ij}$$

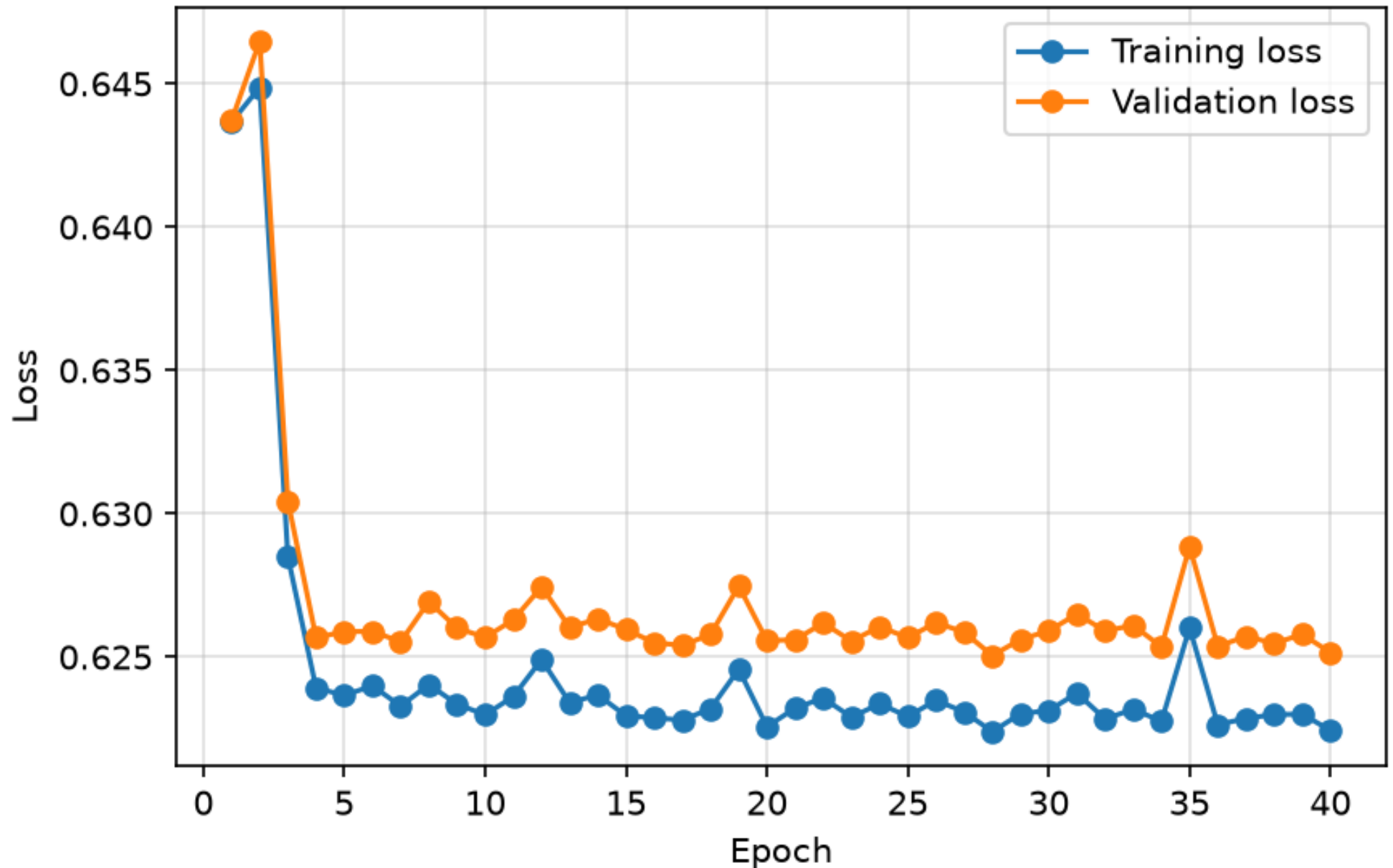
We can find the maximum weight (perfect) matching in G in time $O(n^3)$ ².

Problem: the optimal matching may not be the best matching candidate
 $\Rightarrow$ try the **top k best matchings** in G

²Hungarian Method [Kuh55]

Training loss

Training run - IMDB



Performance of GEDGNN - IMDB

Results over 82665 validation graphs with 8.1 nodes on average.

	MAE	Rounded MAE	Accuracy
GEDGNN	0.516	0.517	56.46%
GEDGNN-matching ($k = 100$)	0.049	–	98.77 %

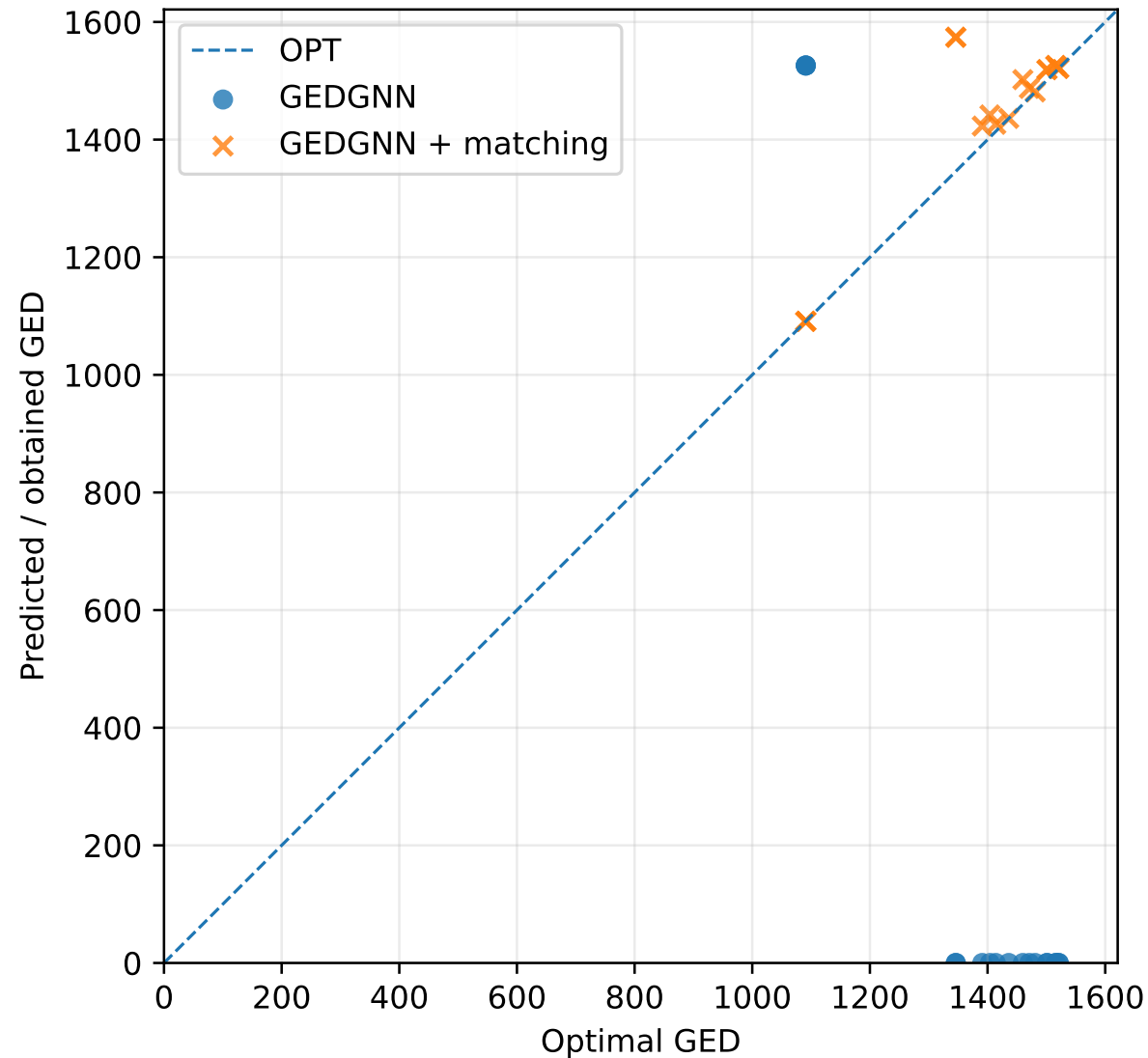
$$\text{MAE} = \frac{1}{N} \left(\sum_i |d_i - d_i^*| \right)$$

$$\text{MAE}_{\text{rounded}} = \frac{1}{N} \left(\sum_i |\text{round}(d_i) - d_i^*| \right)$$

$$\text{acc} = \frac{|\{i : \text{round}(d_i) = d_i^*\}|}{N}$$

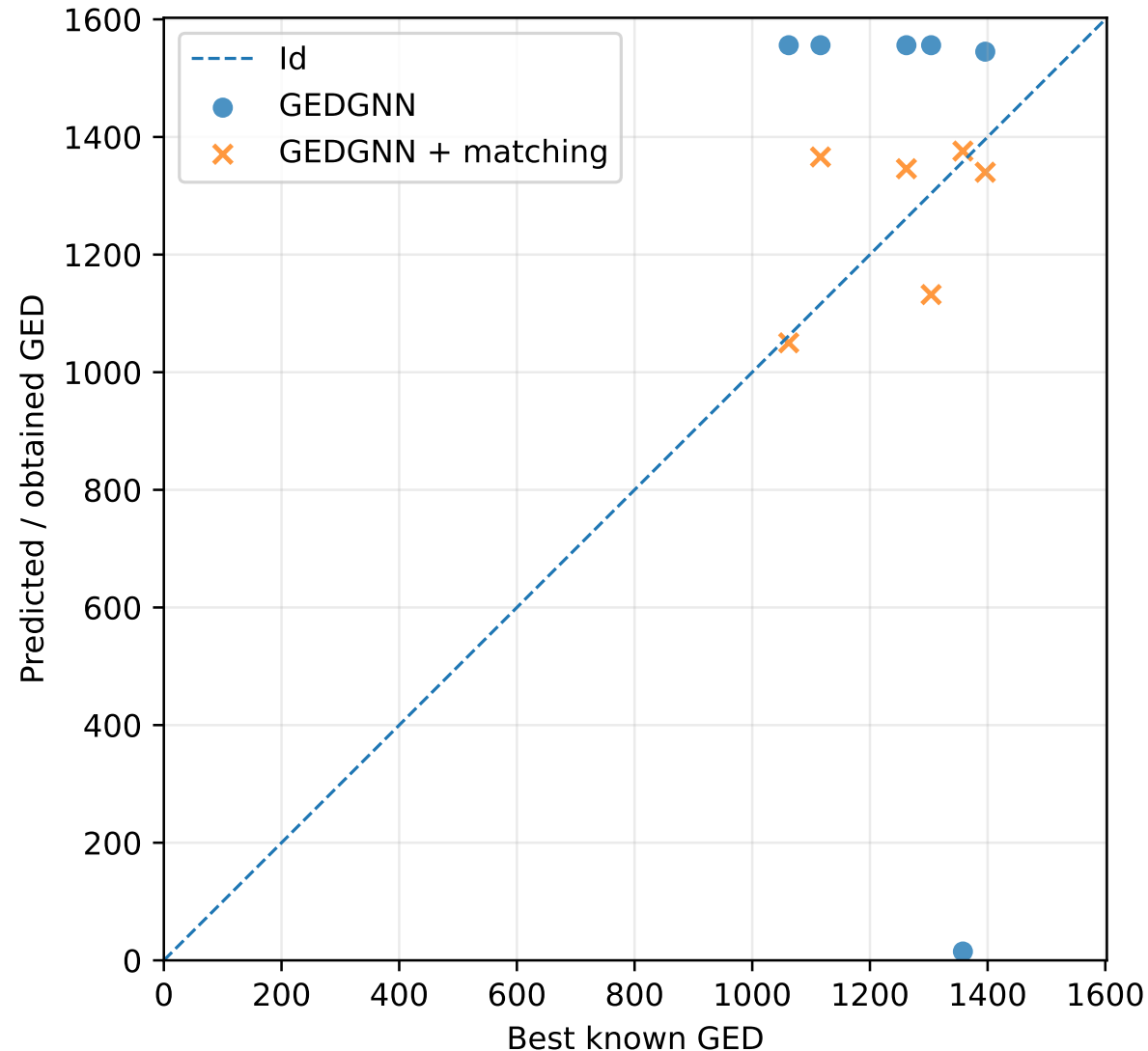
Generalizing to larger instances

- ▶ $GEDGNN_{matching}$ is run with $k = 10$
- ▶ Instances are IMDB instances from [Das+25]



Generalizing to larger instances

- ▶ $GEDGNN_{matching}$ is run with $k = 10$
- ▶ Instances are IMDB instances from [Das+25]



Use of Generative AI

- ▶ Implementation comparisons, especially for GEDGNN
- ▶ Parsing functions for the FORI-GED/GEDGNN data sources
- ▶ Smaller Julia language specific help
- ▶ Help with LaTeX code (Tikz, data extraction)

References I

- [BG20] David B. Blumenthal and Johann Gamper. “On the exact computation of the graph edit distance”. In: **Pattern Recognition Letters** 134 (2020). Applications of Graph-based Techniques to Pattern Recognition, pp. 46–57. ISSN: 0167-8655. DOI: <https://doi.org/10.1016/j.patrec.2018.05.002>.
- [Das+25] Andrea D’ascenzo et al. “Enhancing Graph Edit Distance Computation: Stronger and Orientation-based ILP Formulations”. In: **Proceedings of the VLDB Endowment** 18.11 (2025), pp. 4737–4749.
- [HJ06] Alfred Hero and Derek Justice. “A binary linear programming formulation of the graph edit distance”. In: **IEEE Transactions on Pattern Analysis and Machine Intelligence** 28.8 (2006), pp. 1200–1214.

References II

- [Kuh55] Harold W Kuhn. “The Hungarian method for the assignment problem”. In: **Naval research logistics quarterly** 2.1-2 (1955), pp. 83–97.
- [Ler+17] Julien Lerouge et al. “New binary linear programming formulation to compute the graph edit distance”. In: **Pattern Recognition** 72 (2017), pp. 254–265.
- [Pia+23] Chengzhi Piao et al. “Computing graph edit distance via neural graph matching”. In: **Proceedings of the VLDB Endowment** 16.8 (2023), pp. 1817–1829.